

Job Searching System Using Agentic AI and Resume Analysis

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Aman Pounikar	Akshat Rathor
EN23CS301116	EN23CS301102

Under the Guidance of

Prof. Digendra Singh Rathore

Prof. Ashish Shrivastava



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Job Searching System Using Agentic AI and Resume Analysis

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Aman Pounikar	Akshat Rathor
EN23CS301116	EN23CS301102

Under the Guidance of
Prof. Digendra Singh Rathore
Prof. Ashish Shrivastava



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work “**Job Searching System using Agentic AI and Resume Analysis**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled **Job Searching System using Agentic AI and Resume Analysis** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Department of Engineering completed under the supervision of **Prof. Digendra Singh Rathore** and **Prof. Ashish Shrivastava**, Faculty of Engineering, Mediacaps University Indore is an authentic work.

Further, We declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Aman Pounikar (EN23CS301116)

B.Tech. VI Semester – CSE

Date: _____

Akshat Rathor (EN23CS301102)

B.Tech. VI Semester – CSE

Date: _____

Certificate

We, **Prof. Digendra Singh Rathore** and **Prof. Ashish Shrivastava** certify that the project entitled “**Job Searching System using Agentic AI and Resume Analysis**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Aman Pounikar(EN23CS301116)** and **Akshat Rathor(EN23CS301102)** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Digendra Singh Rathore

Computer Science Department

Medicaps University, Indore

Prof. Kailash C. Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm has always boosted up my morale. We also thank **Prof. (Dr.) Ratnesh Litoriya**, Dean, Faculty of Engineering, Mediacaps University, for giving me a chance to work on this project. We would also like to thank my Head of the Department, **Prof. Kailash C. Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Aman Pounikar: -EN23CS301116

Akhsat Rathor: -EN23CS301102

B.Tech. III Year (B)

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Abstract

This project presents an AI-powered Job Recommendation System that intelligently matches job seekers with relevant job listings based on their resume content. The system accepts a resume in PDF or image format, extracts and preprocesses the text, and automatically detects the candidate's role using TF-IDF keyword extraction. It then fetches real-time job listings from the JSearch API based on the detected role and location preferences provided by the user. A semantic similarity model based on Sentence Transformers (all-MiniLM-L6-v2) computes cosine similarity between the resume and each job description to rank the most relevant positions. The top five matched jobs are presented to the user along with AI-generated explanations produced by the Groq LLaMA 3.1 model, which describes why each job is a good fit. The system is built with a Python backend using FastAPI and a modern React frontend, replacing the initial Streamlit interface for a more polished and scalable user experience. The overall goal of the system is to reduce the time and effort job seekers spend manually searching and filtering job listings by automating the entire matching process end-to-end.

Keywords:

Resume Parsing, Natural Language Processing, TF-IDF, Sentence Transformers, Cosine Similarity, Semantic Matching, Job Recommendation, FastAPI, React, Groq AI, LLaMA, OCR, PyMuPDF, RapidAPI, JSearch

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	viii
	List of tables	ix
	Abbreviations	x
Chapter 1	Introduction	1
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	3
	1.5 Research Design	3
	1.6 Source of Data	4
	1.7 Chapter Scheme	5
Chapter 2	REQUIREMENTS SPECIFICATION	6
	2.1 User Characteris	6
	2.2 Functional Requirements	6
	2.3 Dependencies	7
	2.4 Performance Requirements	7
	2.5 Hardware Requirements	7
	2.6 Constraints & Assumptions	8
Chapter 3	DESIGN	9
	3.1 Algorithm	9
	3.2 Function Oriented Design for procedural approach	9
	3.3 System Design	11
	3.3.1 Data Flow Diagrams (Level 0,Level1)	12
	3.3.2 Activity Diagram	13
	3.3.3 Flow Chart	14

	3.3.4 Class Diagram	15
	3.3.5 ER Diagram	16
	3.3.6 Sequence diagram	17
	3.4 Database Design	17
	3.4.1 Logical Database Design	17
	3.4.2 Physical Database Design	18
Chapter 4	Implementation, Testing, and Maintenance	18
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	18
	4.2 Testing Techniques and Test Plans	20
	4.3 Installation Instructions	21
	4.4 End User Instructions	22
Chapter 5	Results and Discussions	24
	5.1 User Interface Representation (of Respective Project)	24
	5.2 Brief Description of Various Modules of the system	25
	5.3 Snapshots of system with brief detail of each	26
Chapter 6	Summary and Conclusions	31
Chapter 7	Future scope	33
	Appendix	34
	Bibliography	35

List of Figures

Figure No.	Title	Page No.
Fig 3.3.	System Design	11
Fig 3.3.1	Data Flow Diagram (Level 0, Level 1)	12
Fig 3.3.2	Activity Diagram	13
Fig 3.3.3	Flow Chart	14
Fig 3.3.4	Class Diagram	15
Fig 3.3.5	ER Diagram	16
Fig 3.3.6	Sequence Diagram	17
Fig 5.1	Resume Upload Interface	27
Fig 5.2	Resume Parsing and Loading State	27
Fig 5.3	Role Detection Interface	28
Fig 5.4	Input Fields Section	29
Fig 5.5	Job Recommendation Result	30

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
API	Application Programming Interface
PDF	Portable Document Format
UI	User Interface
UX	User Experience
TF-IDF	Term Frequency – Inverse Document Frequency
OCR	Optical Character Recognition
ML	Machine Learning
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
IDE	Integrated Development Environment
CPU	Central Processing Unit
GPU	Graphics Processing Unit
DB	Database
LLM	Large Language Model

Chapter 1: Introduction

1.1 Introduction

In today's competitive job market, finding the right job opportunity that aligns with one's skills and experience is a time-consuming and often overwhelming process. Job seekers are required to manually browse through hundreds of listings across multiple platforms, evaluate relevance, and apply selectively — a process that is both inefficient and prone to missed opportunities.

This project proposes an AI-powered Job Recommendation System that automates this process by intelligently matching a candidate's resume with the most relevant job listings available in real time. The system accepts a resume in PDF or scanned image format, extracts and cleans the text content, identifies the candidate's likely job role, fetches live job listings from the internet, and ranks them based on semantic similarity with the resume. AI-generated explanations are also provided for each matched job to help the candidate understand why a particular listing is relevant to their profile.

The system is built using a Python backend powered by FastAPI, integrating modules for resume parsing, text preprocessing, role detection, job fetching, and semantic matching. The frontend is developed using React, providing a clean, interactive, and user-friendly interface accessible via any modern web browser.

1.2 Literature Review

Several approaches have been explored in the domain of resume parsing and job recommendation systems:

Traditional keyword-based matching systems relied on exact word matches between resumes and job descriptions. While simple to implement, these systems failed to capture semantic meaning, often missing relevant matches where different words conveyed the same skill or experience.

TF-IDF (Term Frequency-Inverse Document Frequency) based approaches improved upon keyword matching by weighing the importance of terms across documents.

However, they still lacked the ability to understand contextual meaning and synonyms.

With the rise of deep learning, transformer-based models such as BERT and its variants introduced semantic understanding into NLP tasks. Models like Sentence-BERT (SBERT) enabled efficient computation of sentence-level embeddings, making semantic similarity tasks practical at scale. The all-MiniLM-L6-v2 model used in this project is a lightweight yet highly effective variant of SBERT that balances performance and speed.

Large Language Models (LLMs) such as GPT and LLaMA have further advanced the field by enabling natural language generation, allowing systems to not just match but also explain recommendations in human-readable form. This project leverages the Groq-hosted LLaMA 3.1 model for generating job match explanations.

Recent industry systems such as LinkedIn's job recommendation engine and Indeed's smart search use similar semantic matching approaches combined with user behavior data, reinforcing the validity of the approach taken in this project.

1.3 Objectives

The key objectives of this project are:

- To develop a system that automatically extracts text from resume files in PDF and image formats
- To preprocess and normalize extracted resume text for use in machine learning pipelines
- To detect the candidate's likely job role from resume content using TF-IDF keyword extraction
- To fetch real-time job listings from external APIs based on detected role and user-specified location
- To rank job listings using semantic similarity computed via Sentence Transformer embeddings
- To generate human-readable explanations for each job match using a large language model
- To provide a clean, modern, and responsive web interface for end users

- To integrate all modules into a cohesive, scalable system using FastAPI and React

1.4 Significance

This project holds significance on multiple levels:

For job seekers: It drastically reduces the time spent manually searching and filtering job listings. By automating role detection and semantic matching, the system surfaces the most relevant opportunities based on the candidate's actual skills and experience rather than manually entered keywords.

For the field of NLP: The project demonstrates a practical application of semantic similarity and transformer-based embeddings in a real-world recommendation task, bridging the gap between academic NLP research and everyday use.

For software development: The project showcases a modern full-stack architecture where a Python AI backend is cleanly separated from a React frontend via a REST API, making the system modular, maintainable, and scalable.

For students and early career professionals: The system is particularly valuable for those who may not know how to optimally search for jobs matching their profile, providing intelligent guidance with minimal effort.

1.5 Research Design

The project follows a pipeline-based system design approach where each module performs a distinct function and passes its output to the next stage. The design is as follows:

- **Input Layer:** User uploads a resume file (PDF or image) via the React frontend
- **Parsing Layer:** Resume Parser extracts raw text using PyMuPDF for PDFs and Pytesseract OCR for images
- **Preprocessing Layer:** Preprocessor cleans and normalizes the extracted text
- **Role Detection Layer:** TF-IDF vectorizer identifies top keywords and constructs a likely job role query

- **Data Fetching Layer:** Job Fetcher queries the JSearch API via RapidAPI using the detected role and location
- **Matching Layer:** Job Matcher encodes resume and job descriptions using Sentence Transformers and computes cosine similarity scores
- **Explanation Layer:** Groq LLaMA 3.1 generates natural language explanations for each top match
- **Presentation Layer:** React frontend displays ranked job cards with scores, explanations, and apply links

1.6 Source of Data

The system uses the following data sources:

- **User-provided data:** Resume files uploaded directly by the user in PDF or image format
- **JSearch API (RapidAPI):** A real-time job search API that aggregates listings from major job platforms including LinkedIn, Indeed, Glassdoor, and others. Queries are made dynamically based on the detected role and user-specified location
- **Pre-trained ML models:** The all-MiniLM-L6-v2 model from the Sentence Transformers library, trained on large-scale text datasets, is used for generating semantic embeddings
- **Groq LLaMA 3.1:** A large language model accessed via the Groq API, used for generating job match explanations

No static datasets were collected or stored. All job data is fetched live at the time of each user request.

1.7 Chapter Scheme

The report is organized into the following chapters:

- **Chapter 1 — Introduction:** Covers the background, literature review, objectives, significance, research design, and data sources of the project
- **Chapter 2 — Requirements Specification:** Details the hardware, software, and library requirements needed to run the system

- **Chapter 3 — Design:** Explains the architecture, module interactions, data flow diagrams, and design decisions
- **Chapter 4 — Implementation, Testing, and Maintenance:** Describes the implementation of each module in detail and covers testing strategies and system maintenance
- **Chapter 5 — Results and Discussions:** Presents the output of the system, test cases, and evaluation of match quality
- **Chapter 6 — Summary and Conclusions:** Summarizes the project outcomes and key findings
- **Chapter 7 — Future Scope:** Discusses potential improvements and future directions for the system

Chapter 2: Requirements Specification

2.1 User Characteristics

The primary users of this system are job seekers looking for relevant opportunities based on their resume. Expected user characteristics are:

- Individuals actively looking for employment across any domain
- Users with a resume prepared in PDF or scanned image format
- Basic familiarity with web browsers and file uploads is sufficient
- Ranges from fresh graduates to experienced professionals
- No programming or technical knowledge required

2.2 Functional Requirements

The system must fulfill the following requirements:

- Accept resume files in PDF and image formats
- Extract, preprocess, and normalize resume text automatically
- Detect the candidate's likely job role using TF-IDF
- Allow users to edit the detected role before searching
- Accept optional location input (country and city)
- Fetch real-time job listings via the JSearch API
- Rank jobs using semantic similarity with the resume
- Return top 5 matched jobs with scores and explanations
- Generate natural language explanations using Groq LLaMA 3.1
- Handle API failures gracefully with fallback strategies

2.3 Dependencies

Python Libraries:

- PyMuPDF (fitz) — PDF text extraction
- OpenCV + pytesseract — Image processing and OCR
- Sentence Transformers — Semantic embeddings
- Scikit-learn — TF-IDF and cosine similarity
- FastAPI + Uvicorn — Backend API server
- Python-multipart — File upload handling
- Groq — LLM API client
- Requests — HTTP calls to JSearch API
- Python-dotenv — Environment variable management

External APIs:

- JSearch API (RapidAPI) — Real-time job listings
- Groq API — LLaMA 3.1 explanation generation

Frontend:

- React + Vite — UI framework and build tool

2.4 Performance Requirements

- Resume text extraction: under 3 seconds
- Role detection (TF-IDF): under 1 second
- Job fetching (JSearch API): under 5 seconds
- Semantic matching: under 10 seconds
- LLM explanation per job: under 3 seconds
- Frontend load time: under 2 seconds
- Current deployment supports one user session at a time

2.5 Hardware Requirements

Minimum:

- Processor: Intel Core i3, 2.0 GHz+
- RAM: 4 GB
- Storage: 2 GB free
- Internet connection required
- OS: Windows 10, macOS, or Linux

Recommended:

- Processor: Intel Core i5, 2.5 GHz+
- RAM: 8 GB
- Storage: 4 GB free
- Broadband: 10 Mbps+
- OS: Windows 10/11, macOS 12+, Ubuntu 20.04+

2.6 Constraints & Assumptions

Constraints:

- Only PDF and image resume formats are supported
- Active internet connection is required at all times
- JSearch and Groq API keys must be valid and active
- System is designed for local deployment only
- OCR accuracy depends on uploaded image quality

Assumptions:

- Resume is properly formatted with skills clearly stated
- Resume language is English
- User has a stable internet connection
- TF-IDF keywords accurately represent the candidate's role

Chapter 3: Design

3.1 Algorithm

The system follows a sequential pipeline algorithm with the following steps:

- Step 1: Accept resume file (PDF or image) from the user
- Step 2: Extract raw text using PyMuPDF for PDFs or Pytesseract OCR for images
- Step 3: Preprocess extracted text — lowercase, remove special characters, strip whitespace
- Step 4: Apply TF-IDF vectorization on clean text to extract top keywords
- Step 5: Construct job search query from top keywords + user-provided location
- Step 6: Fetch live job listings from JSearch API using the constructed query
- Step 7: Preprocess each job description using the same preprocessing pipeline
- Step 8: Encode resume and job descriptions into vector embeddings using Sentence Transformers
- Step 9: Compute cosine similarity between resume embedding and each job embedding
- Step 10: Sort jobs by similarity score in descending order, select top 5
- Step 11: For each top job, generate a natural language explanation using Groq LLaMA 3.1
- Step 12: Return ranked jobs with scores and explanations to the React frontend for display

3.2 Function Oriented Design

The system is designed using a modular, function-oriented approach where each module is responsible for a single well-defined task. The key functions are:

resume_parser.py

- `extract_text_from_pdf(file_path)` — Opens a PDF and extracts text page by page using PyMuPDF
- `extract_text_from_image(file_path)` — Reads an image, converts to grayscale, and runs Pytesseract OCR

preprocessor.py

- `preprocess_text(text)` — Converts text to lowercase, removes special characters, and strips extra whitespace

app.py / api.py

- `detect_role(text)` — Applies TF-IDF on resume text and extracts top keywords to form a job role query

job_fetcher.py

- `fetch_jobs(query)` — Sends a GET request to JSearch API with the search query and returns a list of job objects

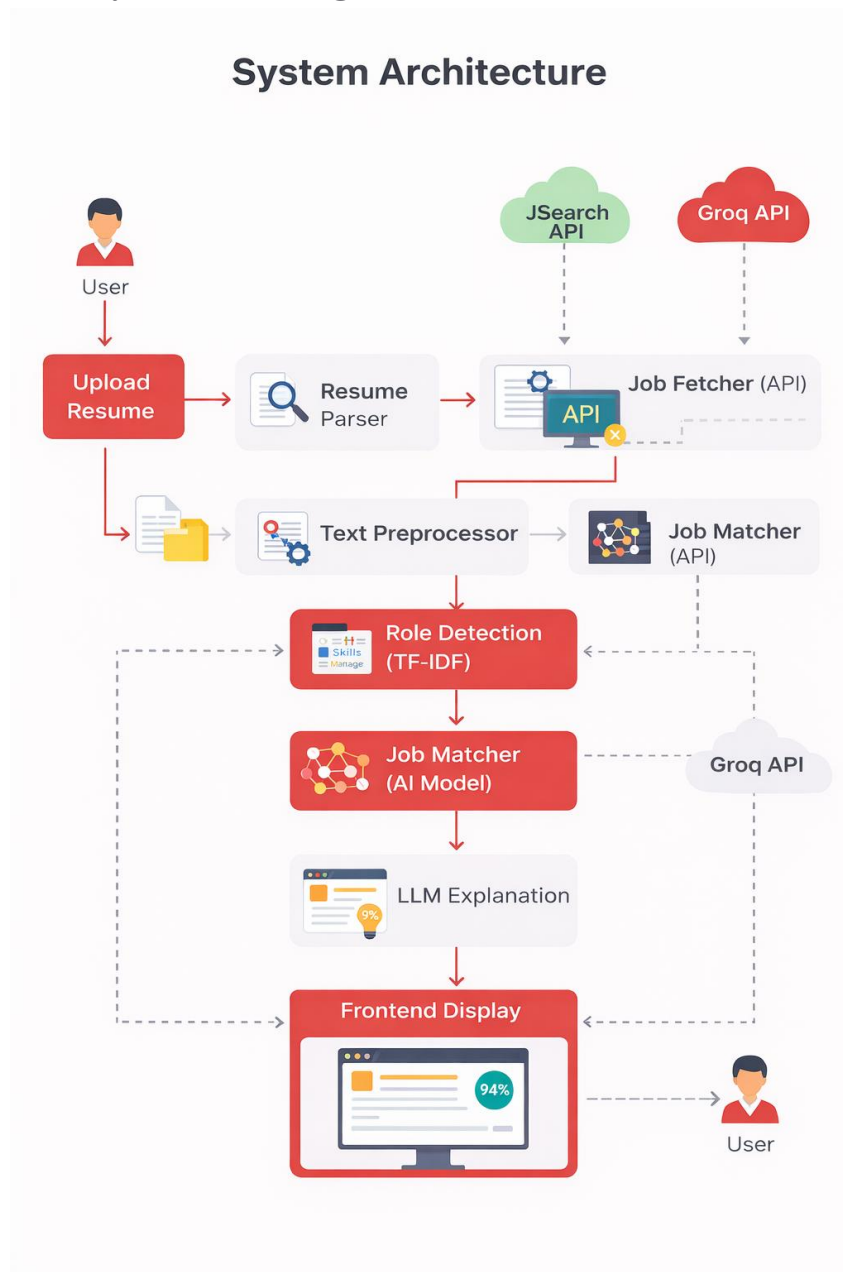
job_matcher.py

- `match_jobs(resume_text, jobs)` — Encodes resume and job descriptions, computes cosine similarity, and returns top 5 ranked jobs

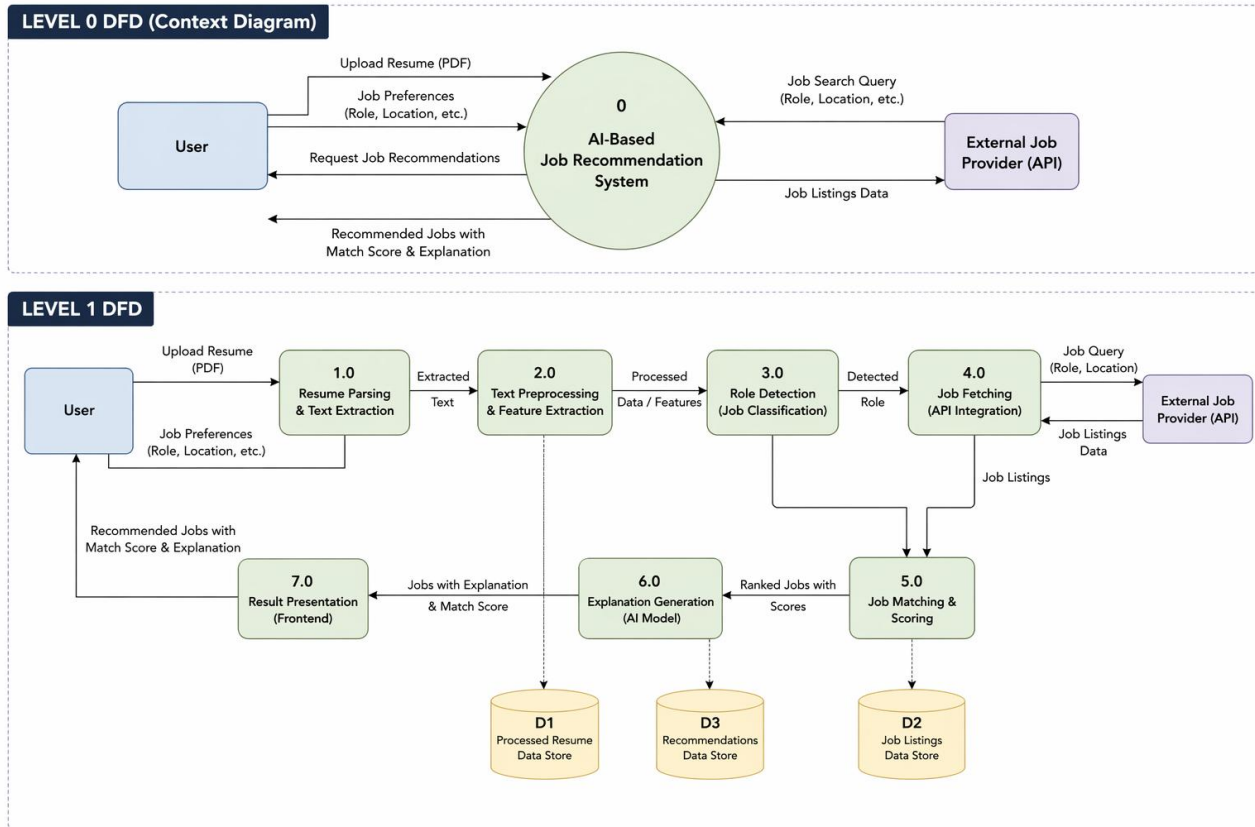
api.py

- `parse_resume(file)` — FastAPI endpoint that handles resume upload, parsing, preprocessing, and role detection
- `match_jobs_endpoint(...)` — FastAPI endpoint that handles job fetching, matching, and explanation generation
- `generate_explanation(resume_text, job)` — Calls Groq LLaMA 3.1 to generate a match explanation for a job

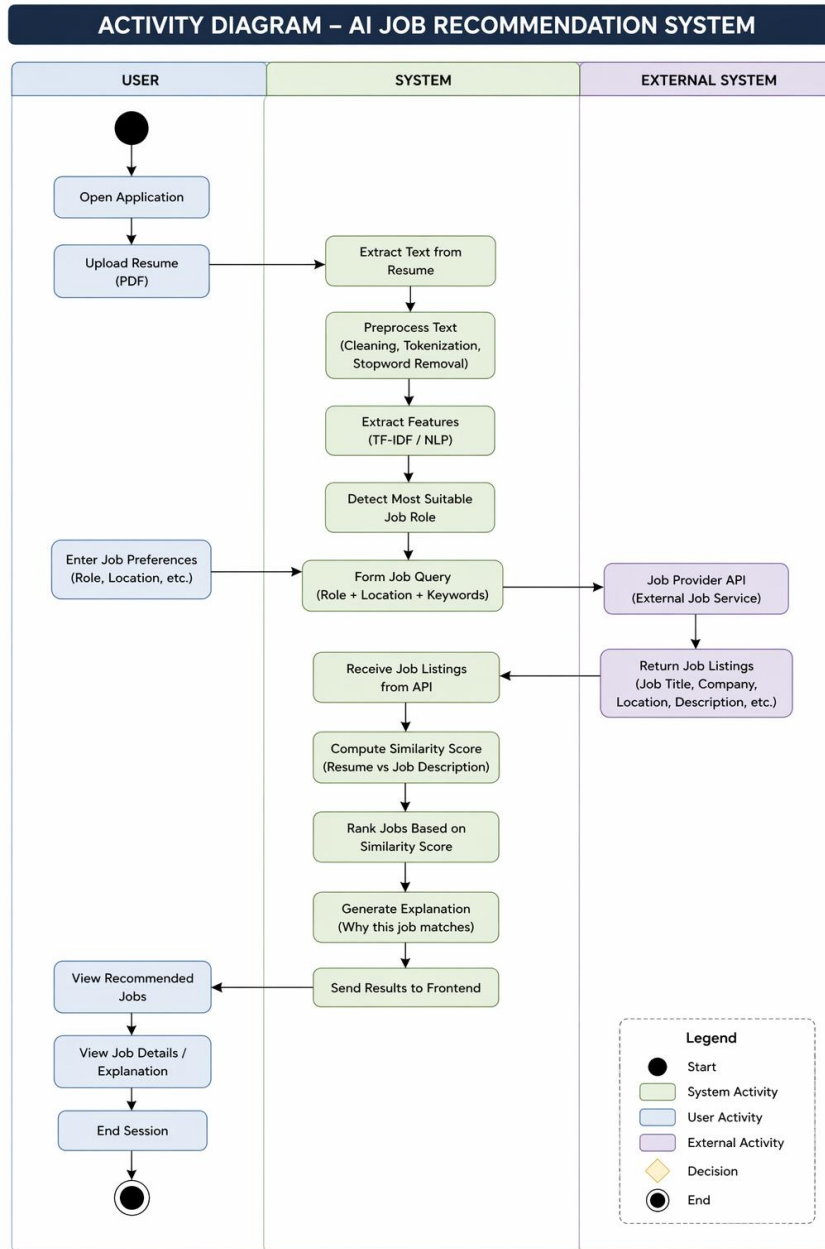
3.3 System Design



3.3.1 Data Flow Diagrams (Level 0, Level 1)

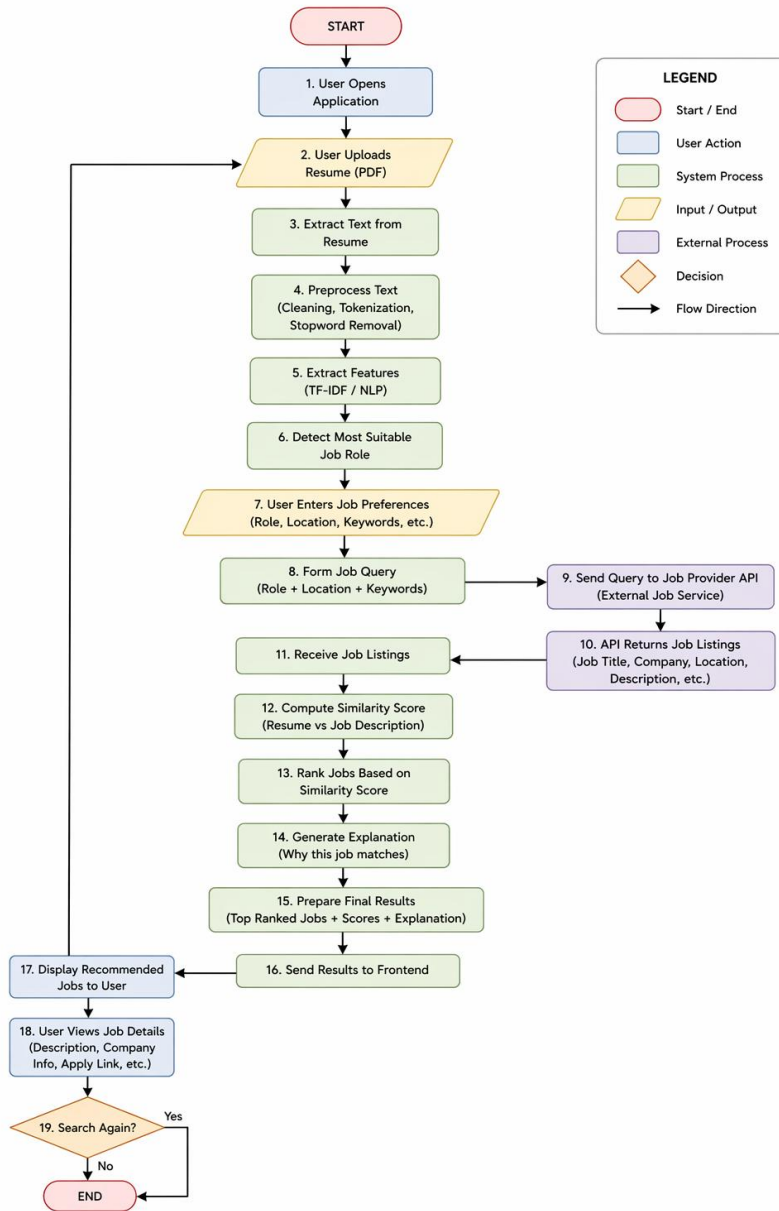


3.3.2 Activity Diagram



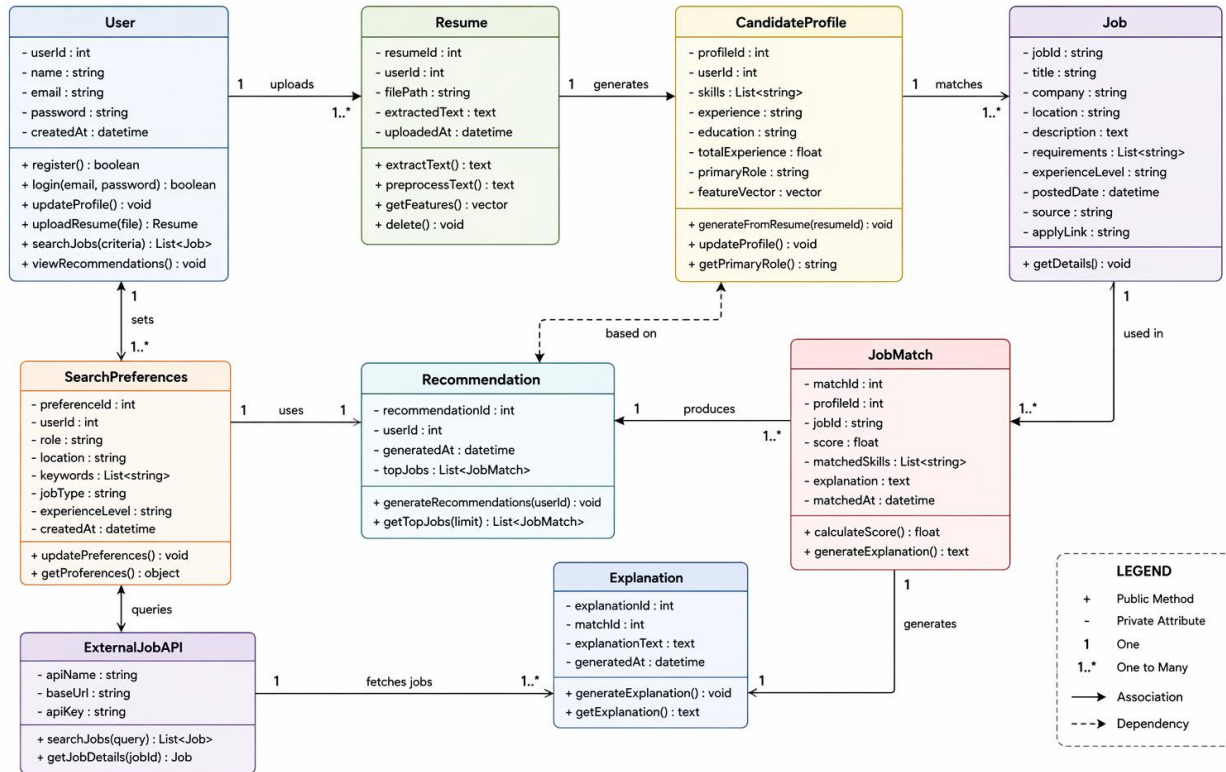
3.3.3 Flow Chart

FLOW CHART – AI JOB RECOMMENDATION SYSTEM



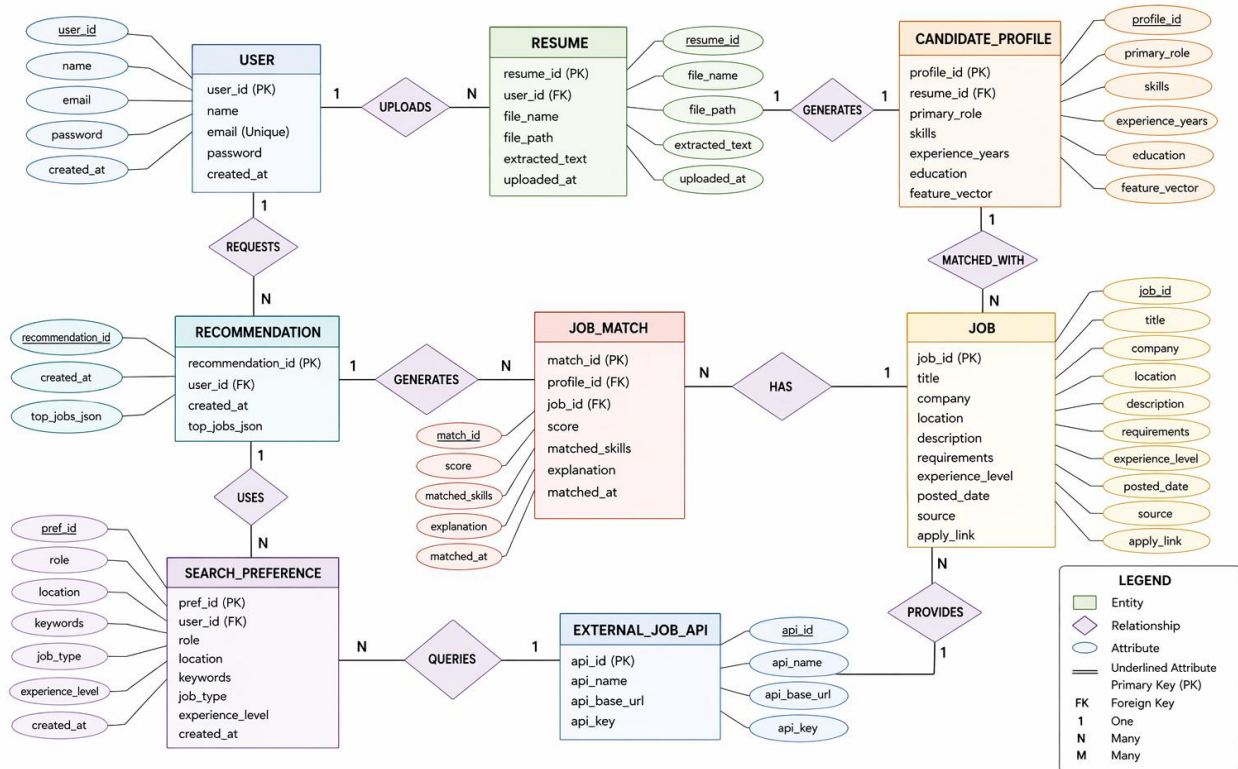
3.3.4 Class Diagram

CLASS DIAGRAM – AI JOB RECOMMENDATION SYSTEM

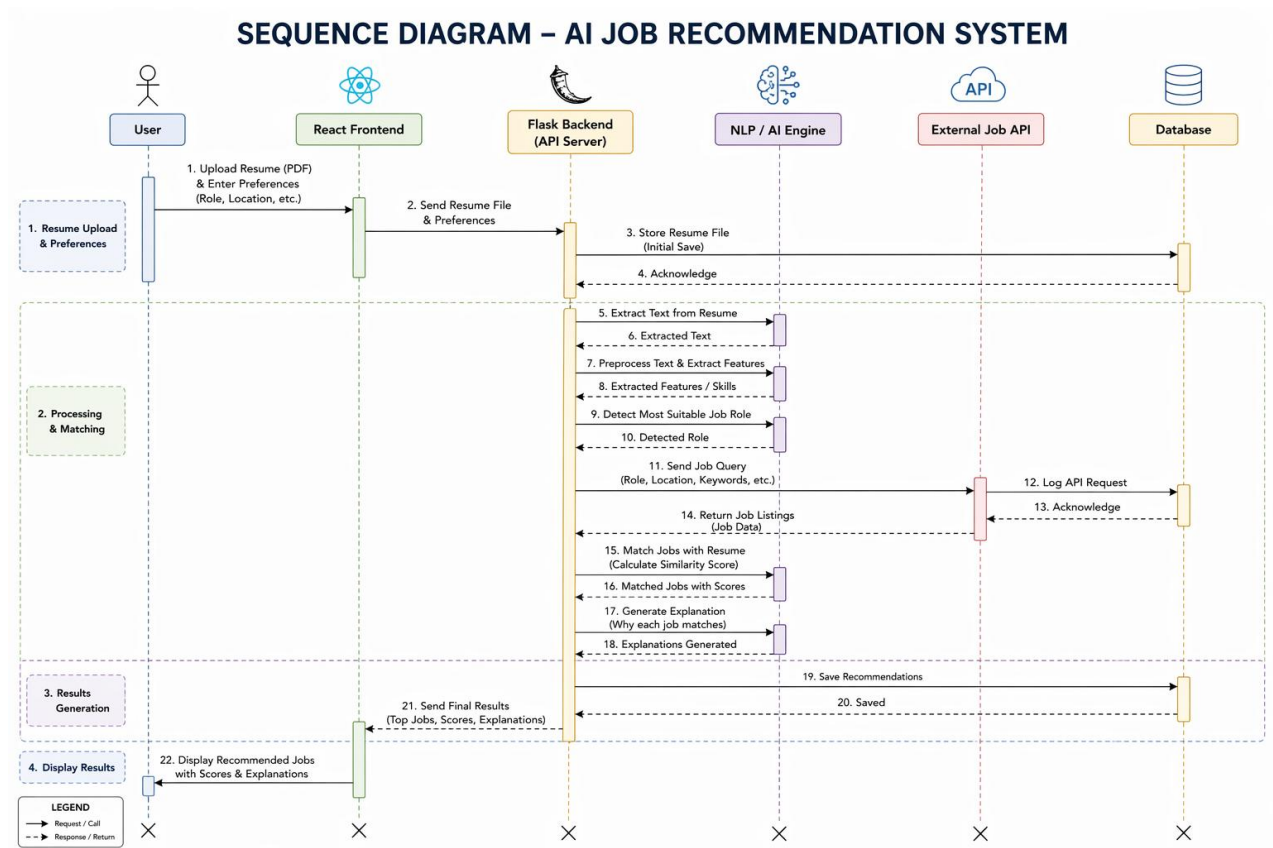


3.3.5 ER Diagram

ER DIAGRAM – AI JOB RECOMMENDATION SYSTEM



3.3.6 Sequence Diagram



3.4 Database Design

This system does not use a persistent database. All data is processed in-memory during each user session. Job listings are fetched live from the JSearch API and are not stored. Resume content is temporarily held in memory during processing and discarded after the session ends.

3.4.1 Logical Database Design

Since no database is used, there is no logical schema to define. The data entities involved — resume text, job listings, match scores, and explanations — exist only as in-memory Python objects during runtime.

3.4.2 Physical Database Design

No physical database is implemented in the current version of the system. Future versions may incorporate a database to store user profiles, search history, and cached job listings for improved performance.

Chapter 4: Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies

The following languages, tools, and technologies were used to implement the system:

Programming Languages:

- Python 3.11+ — Used for all backend modules including parsing, preprocessing, matching, and API development
- JavaScript (ES6+) — Used for the React frontend
- HTML/CSS — Used for frontend structure and styling

IDEs and Editors:

- PyCharm — Primary IDE for Python backend development
- Visual Studio Code — Used for React frontend development

Frameworks and Libraries:

- FastAPI — Lightweight Python web framework for building REST APIs
- React — JavaScript library for building the user interface
- Vite — Frontend build tool and development server
- Sentence Transformers — Library for generating semantic text embeddings
- Scikit-learn — Used for TF-IDF vectorization and cosine similarity
- PyMuPDF (fitz) — PDF text extraction
- OpenCV — Image processing for scanned resumes
- Pytesseract — OCR engine for extracting text from images
- Groq SDK — Python client for accessing LLaMA 3.1 via Groq API

External APIs and Services:

- JSearch API (RapidAPI) — Provides real-time job listings
- Groq API — Provides access to LLaMA 3.1 for explanation generation

Other Tools:

- Git — Version control
- dotenv — Secure management of API keys via environment variables
- Uvicorn — ASGI server for running the FastAPI backend
- npm — Package manager for frontend dependencies

4.2 Testing Techniques and Test Plans

The system was tested using the following approaches:

Unit Testing:

- Each module was tested independently before integration
- `extract_text_from_pdf()` was tested with multiple resume formats to verify accurate text extraction
- `preprocess_text()` was tested with noisy inputs containing symbols, numbers, and mixed casing
- `fetch_jobs()` was tested with various role and location queries to verify API response handling
- `match_jobs()` was tested with different resumes to verify correct similarity scoring and ranking

Integration Testing:

- The full pipeline was tested end-to-end by uploading a real resume and verifying that the correct role was detected, relevant jobs were fetched, and accurate matches were returned
- FastAPI endpoints were tested using the auto-generated Swagger UI at `/docs`

Frontend Testing:

- The React UI was tested manually across different browsers
- File upload, drag-and-drop, role editing, and job card rendering were all verified
- Error states (no jobs found, backend unavailable) were tested to ensure graceful handling

Test Cases:

Test Case	Input	Expected Output	Result
PDF Upload	Valid resume PDF	Extracted text returned	Pass
Image Upload	Scanned resume image	OCR text extracted	Pass
Role Detection	Clean resume text	Relevant role keyword	Pass
Job Fetch	Role + location query	List of job objects	Pass
Job Matching	Resume + job list	Top 5 ranked jobs	Pass
Fallback Query	Empty job results	Broader query retried	Pass
API Failure	Invalid API key	Error handled gracefully	Pass

4.3 Installation Instructions

Follow these steps to set up and run the system locally:

Prerequisites:

- Python 3.11 or above
- Node.js 18 or above
- npm 9 or above
- Tesseract OCR installed on the system
- Valid RapidAPI key (JSearch API) and Groq API key

Step 1 — Clone or download the project:

Download and extract the project folder to your local machine

Step 2 — Set up environment variables:

Create a .env file in the Modules folder with the following:

GROQ_API_KEY=your_groq_api_key

RAPIDAPI_KEY=your_rapidapi_key

Step 3 — Install Python dependencies:

pip install fastapi uvicorn python-multipart groq python-dotenv

pip install pymupdf opencv-python pytesseract sentence-


```
transformers  
pip install scikit-learn requests
```

Step 4 — Start the backend:

```
cd "Project main folder/Modules"  
python -m uvicorn api:app --reload
```

Step 5 — Install frontend dependencies:

```
cd "job-matcher"  
npm install
```

Step 6 — Start the frontend:

```
npm run dev
```

Step 7 — Open the application:

Open <http://localhost:5173> in your browser

4.4 End User Instructions

Follow these steps to use the AI Job Matcher system:

- **Step 1 — Open the application** by navigating to <http://localhost:5173> in any modern web browser
- **Step 2 — Upload your resume** by clicking the upload zone or dragging and dropping your PDF resume file into it
- **Step 3 — Wait for parsing** — the system will automatically extract text from your resume and detect your job role, which will appear on screen within a few seconds
- **Step 4 — Review detected role** — the AI-detected role will be shown. You can edit it manually if needed to better reflect your target position
- **Step 5 — Enter location** — optionally enter your preferred country and city to get location-specific job results

- **Step 6 — Click Find Matching Jobs** — the system will fetch live job listings and rank them based on your resume
- **Step 7 — Review results** — the top 5 matched jobs will appear as cards showing the job title, company name, match score, and an AI-generated explanation of why it is a good fit
- **Step 8 — Apply** — click the Apply Now button on any job card to be redirected to the job listing and apply directly

Chapter 5: Results and Discussions

5.1 User Interface Representation (of Respective Project)

The user interface of the system is developed using React and is designed to provide a clean, intuitive, and responsive experience. The interface follows a structured workflow that guides the user from resume upload to job recommendation.

The application is divided into the following UI sections:

- **Landing / Hero Section** — Displays the title and purpose of the application
- **Resume Upload Section** — Allows users to upload their resume using drag-and-drop or file selection
- **Parsing Feedback Section** — Displays loading indicators while the resume is being processed
- **Role Detection Section** — Shows the automatically detected job role based on resume content
- **Input Fields Section** — Allows users to modify role and enter preferences such as country and city
- **Results Section** — Displays recommended jobs in a structured card format

Each job card contains:

- Job title
- Company name
- Match score (percentage)
- AI-generated explanation
- Apply link

The UI is responsive and works across different screen sizes, ensuring usability on desktops and laptops.

5.2 Brief Description of Various Modules of the System

The system is divided into the following modules:

1. Resume Parsing Module

- Extracts text from PDF resumes using PyMuPDF
- Uses OCR (Tesseract + OpenCV) for scanned resumes
- Handles both digital and image-based resumes

2. Text Preprocessing Module

- Removes noise such as special characters and stopwords
- Normalizes text (lowercasing, cleaning)
- Prepares text for feature extraction

3. Feature Extraction Module

- Uses TF-IDF and Sentence Transformers
- Converts resume text into numerical vectors
- Captures semantic meaning of text

4. Role Detection Module

- Identifies the most relevant job role based on resume content
- Extracts key terms and keywords

5. Job Fetching Module

- Retrieves job listings using JSearch API
- Uses role and location as input
- Handles fallback queries if no jobs are found

6. Job Matching Module

- Computes similarity between resume and job descriptions
- Uses cosine similarity for ranking
- Selects top matching jobs

7. Explanation Generation Module

- Uses LLaMA 3.1 via Groq API
- Generates explanations for why a job matches the resume

8. Frontend Module

- Built using React
- Handles user interaction and displays results
- Provides smooth UI transitions and feedback

5.3 Snapshots of System with Brief Detail of Each

The following screenshots represent different stages of the system:

Figure 5.1: Resume Upload Interface

Shows the drag-and-drop functionality where users can upload their resume in PDF format.

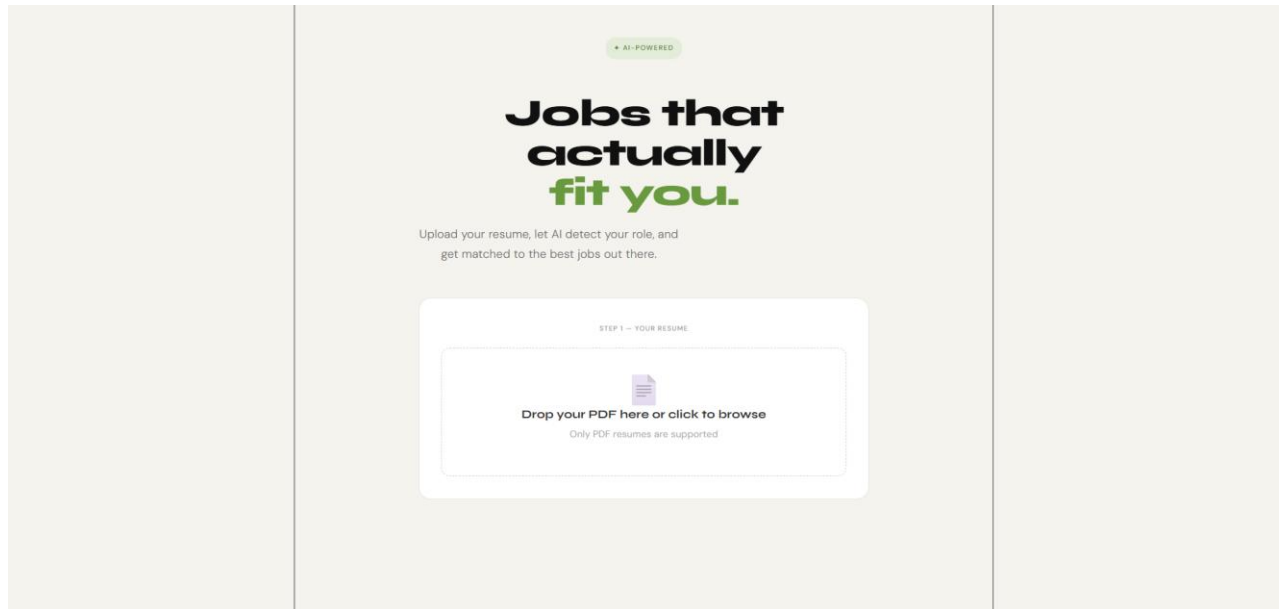


Figure 5.2: Resume Parsing and Loading State

Displays the loading animation while the system processes the resume and extracts text.

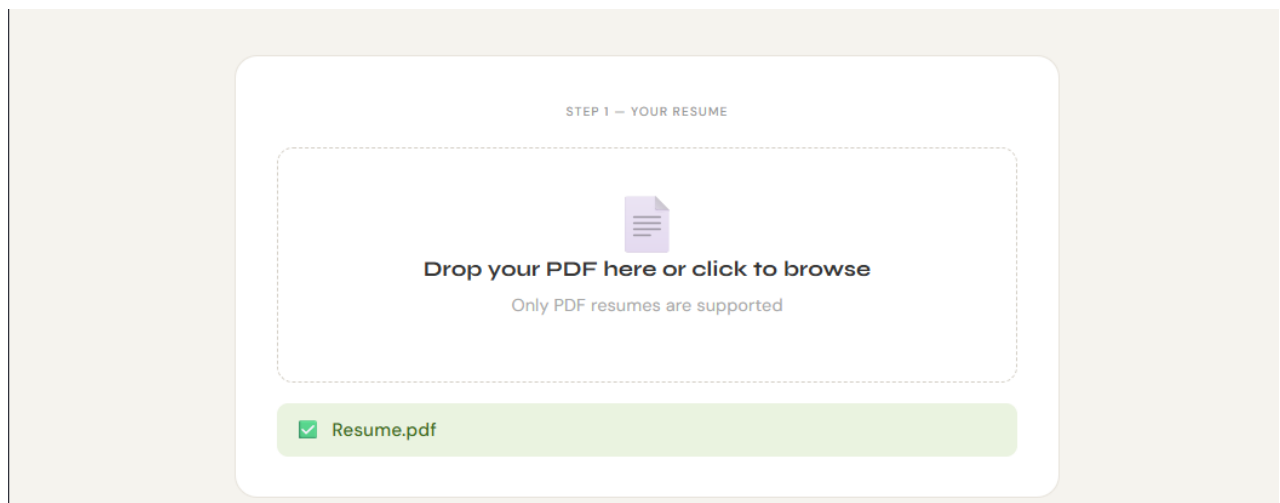
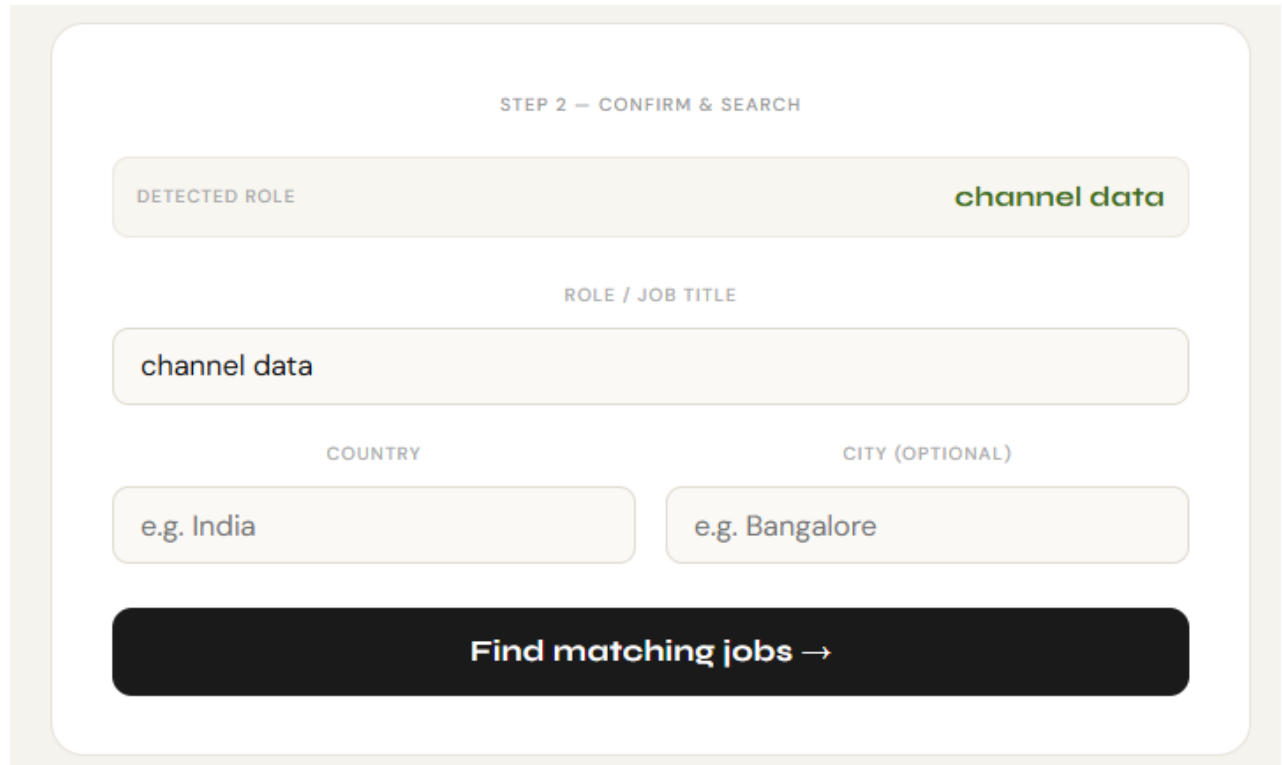


Figure 5.3: Role Detection Interface

Shows the automatically detected job role along with editable input fields for user preferences.



The image shows a web form titled "STEP 2 — CONFIRM & SEARCH". It contains several input fields for job search criteria. The first field, labeled "DETECTED ROLE", contains the text "channel data". The second field, labeled "ROLE / JOB TITLE", also contains "channel data". Below these are two fields for location: "COUNTRY" with the example "e.g. India" and "CITY (OPTIONAL)" with the example "e.g. Bangalore". At the bottom is a large black button with the text "Find matching jobs →".

STEP 2 — CONFIRM & SEARCH

DETECTED ROLE **channel data**

ROLE / JOB TITLE

channel data

COUNTRY CITY (OPTIONAL)

e.g. India e.g. Bangalore

Find matching jobs →

Figure 5.4: Input Fields Section

Allows users to enter country and city for location-based job filtering.

STEP 2 — CONFIRM & SEARCH

DETECTED ROLE **channel data**

ROLE / JOB TITLE

channel data

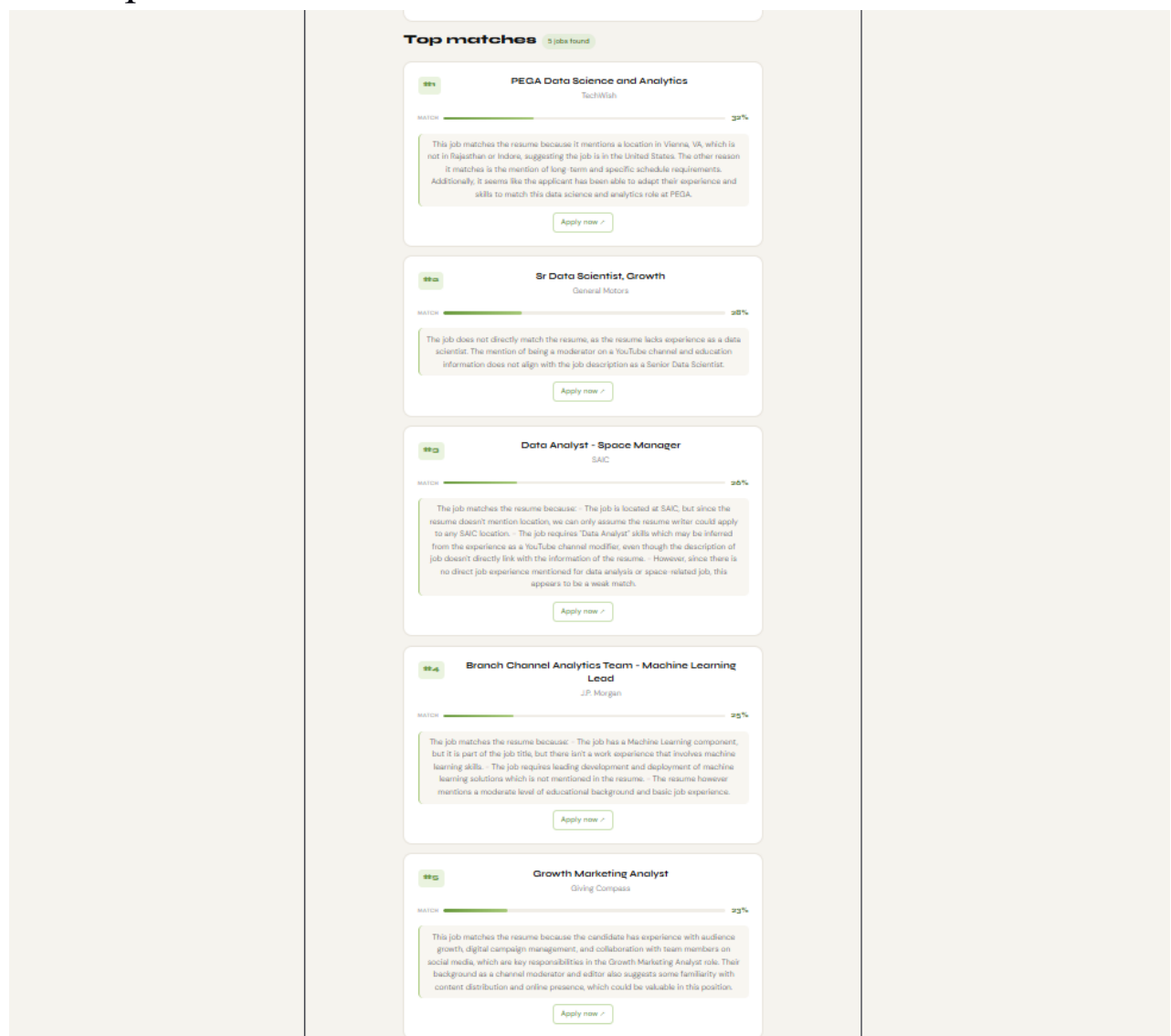
COUNTRY CITY (OPTIONAL)

e.g. India e.g. Bangalore

Find matching jobs →

Figure 5.5: Job Recommendation Results

Displays the top matched jobs in card format with match score and explanation.



Chapter 6: Summary and Conclusions

Summary:-

This project presents an AI-Based Job Recommendation System that automates the process of job searching using resume analysis. The system integrates Natural Language Processing (NLP), Machine Learning techniques, and modern web technologies to provide personalized job recommendations.

The system begins by allowing users to upload their resume in PDF format. The resume is then processed using text extraction techniques, including OCR for scanned documents. The extracted text is cleaned and preprocessed to remove noise and prepare it for analysis.

Feature extraction is performed using TF-IDF and sentence embeddings, enabling the system to represent the resume in a structured numerical format. Based on this representation, the system identifies the most relevant job role and uses it to fetch job listings through an external API.

The fetched job listings are then compared with the resume using similarity algorithms, and the top matching jobs are ranked based on their relevance. Additionally, the system generates AI-based explanations for each recommendation, helping users understand why a particular job is suitable.

The frontend is developed using React, providing a modern and responsive user interface, while the backend is implemented using FastAPI, ensuring efficient processing and API communication.

Conclusions:-

The developed system successfully demonstrates the application of AI and NLP techniques in solving real-world problems related to job searching and recommendation.

The key conclusions drawn from the project are:

- Resume-based job recommendation significantly improves the relevance of job search results
- NLP techniques such as TF-IDF and embeddings are effective for analyzing and comparing textual data
- Integration of external job APIs enables access to real-time job listings
- AI-generated explanations enhance user trust and transparency in recommendations
- A modern frontend improves user interaction and overall experience

The system meets its primary objective of providing an intelligent, efficient, and user-friendly job recommendation platform.

Overall, the project highlights the potential of combining machine learning, natural language processing, and web development to build scalable and practical applications.

Chapter 7: Future Scope

The current system provides an effective solution for resume-based job recommendation using AI and NLP techniques. However, there are several areas where the system can be further enhanced to improve functionality, scalability, and user experience.

The possible future improvements are as follows:

- **User Authentication and Profiles**
Implement a login system to allow users to create accounts, save resumes, and track job recommendations over time.
- **Database Integration**
Introduce a database system (such as MySQL or MongoDB) to store user data, job history, and preferences for better personalization.
- **Advanced AI Models**
Replace or enhance current matching techniques with advanced deep learning models or fine-tuned transformer-based models for improved accuracy.
- **Resume Improvement Suggestions**
Provide AI-based feedback on resumes, suggesting improvements in skills, formatting, and content.
- **Multi-language Support**
Extend the system to support resumes in multiple languages using multilingual NLP models.
- **Job Application Tracking System**
Allow users to track applied jobs, application status, and interview progress.
- **Mobile Application Development**
Develop a mobile version of the application for better accessibility.
- **Real-time Notifications**
Notify users about new job opportunities matching their profile.

These enhancements will make the system more robust, scalable, and closer to real-world job recommendation platforms.

Appendix

The appendix includes additional supporting material related to the project.

Appendix A: System Diagrams

- Data Flow Diagram (Level 0 and Level 1)
- Activity Diagram
- Flowchart
- Class Diagram
- ER Diagram
- Sequence Diagram

Appendix B: Sample Inputs and Outputs

- Sample resume input (PDF)
- Extracted text from resume
- Sample job listings
- Matched job output

Appendix C: API Details

- JSearch API configuration
- Groq API usage for explanation generation

Bibliography

The following references were used during the development of this project:

1. Python Software Foundation, *Python Documentation*, <https://docs.python.org>
2. React Documentation, <https://react.dev>
3. FastAPI Documentation, <https://fastapi.tiangolo.com>
4. Scikit-learn Documentation, <https://scikit-learn.org>
5. Sentence Transformers Documentation, <https://www.sbert.net>
6. PyMuPDF Documentation, <https://pymupdf.readthedocs.io>
7. Tesseract OCR Documentation, <https://github.com/tesseract-ocr/tesseract>
8. RapidAPI – JSearch API Documentation, <https://rapidapi.com>
9. Groq API Documentation (LLaMA Models), <https://groq.com>
10. Research papers on NLP and job recommendation systems.